An Empirical Study of MuRIL-based architectures for Hindi Health Domain NER

***Abstract*—Named Entity Recognition (NER) is a fundamental task in Natural Language Processing that aims to identify and classify named entities in text into predefined categories. In this work, we present an experimental comparison of MuRIL-based architectures for Hindi health-domain NER. A strong baseline model consisting of MuRIL with a Conditional Random Field (CRF) decoding layer, and a stacked architecture that adds together MuRIL embeddings with BiLSTM, token-level attention, and convolutional layers for capturing sequential and local contextual patterns, followed by CRF-based structured decoding are evaluated. We also include additional MuRIL variants such as frozen encoder training, softmax-based decoding, and LoRA-based fine-tuning. Experiments on the Hindi Health Dataset (HHD) show that MuRIL+CRF provides a strong baseline, while the stacked model achieves marginally increased performance with modest category-wise improvements for certain medical entities.**

***Index Terms*—Named Entity Recognition, Hindi NER, Healthcare NLP, Sequence labeling, Neural Network, Transformer Fine-Tuning, MuRIL**

I. Introduction

Named Entity Recognition (NER) is a fundamental task in extraction of information that aims to identify and classify named entities from unstructured text into predefined semantic categories. These categories are generally entities such as person, location, organization, numerical expressions, dates, and domain-specific entities depending on the application. Formally, NER is a two-stage process consisting of (i) detecting spans of text that correspond to named entities and (ii) assigning each detected span an appropriate entity label [1]. A named entity may consist of a single token or a sequence of multiple tokens which together convey a specific semantic meaning within the text.

Consider the following Hindi sentences:

डॉक्टर ने बताया कि मरीज को गुर्दे की पथरी और तेज़ पेट दर्द की समस्या है। इलाज के लिए उसे एंटीबायोटिक दवा और पर्याप्त पानी पीने की सलाह दी गई।

NER system first identifies candidate named entities from the medical text such as:

डॉक्टर, मरीज, गुर्दे की पथरी, तेज़ पेट दर्द, एंटीबायोटिक दवा, पानी

In the subsequent classification stage, these entities are mapped to domain-specific semantic categories including ***PERSON***, ***DISEASE***, ***SYMPTOM*** and ***CONSUMABLE***.

Correct identification and classification of such medical entities allow the transformation of unstructured clinical records into structured information, which is important for healthcare applications such as clinical decision systems, medical information capturing, automated health record analysis, and disease surveillance.

In recent era, deep neural language models have significantly advanced the state-of-the-art in NER and other NLP tasks by learning rich contextual representations of text. Pre-trained language models such as BERT [2], RoBERTa [3], ELECTRA [4], ALBERT [5], and T5 [6] have shown remarkable performance for English-language tasks. For multilingual settings, models such as mBERT, XLM [7], and XLM-R [8] have been proposed. However, despite being trained on data from multiple languages, these multilingual models often give inferior results on Indian languages due to limited representation in training corpora and vocabulary, as well as the linguistic complexity of these languages.

To address this limitation, Khanuja et al. (2021) introduced *MuRIL* (Multilingual Representations for Indian Languages) [9], a transformer-based language model specifically trained on Indian languages and English. MuRIL adopts a 12-layer encoder architecture similar to BERT-base and is trained using Masked Language Modeling (MLM) and Translation Language Modeling (TLM) objectives. Empirical studies have shown that MuRIL consistently outperforms general-purpose multilingual models on several Indian language benchmarks.

Currently Hindi language NER research is focused only on general-domain entities such as person, location, and organization, domain-specific NER—particularly in the healthcare domain—remains underexplored. Medical text in Hindi presents unique challenges, including long compound expressions, high lexical variation, multi-word disease and symptom mentions, and limited annotated resources. Accurate identification of medical entities such as diseases, symptoms, consumables, and patient-related mentions is important for applications such as clinical decision support, healthcare information extraction, and public health analytics.

In this work, we study a stacked hybrid neural architecture for Hindi medical-domain NER along with baseline MuRIL–CRF based model, evaluated on the Hindi Health Dataset (HHD). Building upon prior MuRIL-based NER models, we developed a stacked architecture that adds MuRIL embeddings along with Bi-directional Long Short-Term Memory (BiLSTM), token-level attention, convolutional neural networks (CNN), and a Conditional Random Field (CRF) decoding layer.

We study and compare several MuRIL based frameworks like one that completely relies on transformer-based representations followed by a CRF layer, another that explicitly enriches token-level representations through hierarchy based

contextualization and pattern modeling before sequence-level decoding. The architecture is trained end-to-end without manual feature engineering or handcrafted linguistic rules, using BIO tagging for entity annotation.

The main contributions of this work are summarized as follows:

- A systematic and detailed empirical study of multiple MuRIL-based neural architectures for Hindi named entity recognition in the healthcare domain is presented.
- We focus on medical-domain Hindi NER using the Hindi Health Domain dataset and do a detailed intuitive and experimental metrics analysis.

II. Problem Definition

Given a Hindi sentence represented as a sequence of tokens $X = \{x_1, x_2, \ldots, x_T\}$, the objective of Named Entity Recognition (NER) is to predict a corresponding sequence of labels $Y = \{y_1, y_2, \ldots, y_T\}$, where each y_i belongs to a predefined tag set. In this work, the tag set consists of health-related entity types including *Disease*, *Symptom*, *Consumable*, and *Person*, along with the non-entity label *O*. We formulate the task as a sequence labeling problem under the BIO tagging scheme, where *B* denotes the beginning of an entity span, *I* denotes a token inside an entity span, and *O* denotes a non-entity token.

III. Related Work

The emergence of deep learning techniques represented a major change in Hindi NER research by reducing dependency on manual feature engineering. Neural network-based models demonstrated the ability to automatically learn precise representations directly from raw text. Early deep learning approaches utilized distributed word embeddings such as GloVe and Skip-gram, which were put into bidirectional Long Short-Term Memory (BiLSTM) networks to model contextual relations [10].Yet, these models often relied on additional linguistic information such as part-of-speech tags to improve performance.

Further research gave importance to character-level modeling to better capture and understand the morphological richness and orthographic variations of Hindi. Gupta et al. (2018) proposed a deep learning-based NER system that added together character-level and word-level embeddings with a Gated Recurrent Unit (GRU) layer for code-mixed Indian social media text [11]. Murthy (2017) introduced a neural architecture that added together CNN-based character representations with word embeddings and fed them into a BiLSTM followed by a softmax classifier [12]. Further improvements incorporated CRF layers on top of recurrent networks to model label dependencies and improve sequence-level prediction accuracy [13], [14].

Many studies explored stronger representations by combining word-level, character-level within BiLSTM-CRF architectures [13]. Later on advancements such as denoising autoencoders were introduced to improve effectiveness, especially for noisy and informal text [15]. These neural architectures have consistently beaten traditional machine learning approaches for Hindi NER.

More recently, pretrained transformer-based language models have achieved state-of-the-art results across various NLP tasks, including NER. Multilingual models such as mBERT, XLM-R [8], and mT5 [23] were trained on large-scale multilingual corpora, while IndicBERT [16] and MuRIL [9] were specifically designed for Indian languages. Although mBERT has been successfully applied to NER tasks in multiple languages such as Portuguese [17], Spanish [18], Chinese [19], Russian [20], German [21], and Slavic languages [22], systematic exploration of these models for Hindi NER, particularly in domain-specific settings, remains limited.

Inspired by these observations, our work attempts to build upon prior research and study a deeper and richer architecture for Hindi NER in the healthcare domain along with baseline MuRIL–CRF architecture. We expand the MuRIL-based framework [24] by adding together BiLSTM for sequential context modeling, a token-level attention mechanism to emphasize main tokens, convolutional neural networks to understand and capture local n-gram patterns, and a CRF layer for structured prediction. This stacked MuRIL + BiLSTM + Attention + CNN + CRF architecture is specifically designed to address the challenges of medical entity recognition in Hindi and showcases huge competition to the baseline MuRIL+CRF framework on the Hindi Health Domain (HHD) dataset [25].

IV. Model Archiectures

This section describes two neural architectures for Hindi health-domain named entity recognition: a baseline MuRIL + CRF model and stacked architecture that extends MuRIL representations with additional sequence modeling and feature extraction layers.

A. MuRIL + CRF Baseline Architecture

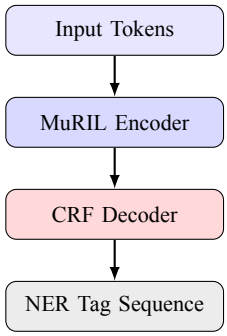

Fig. 1. Baseline MuRIL + CRF architecture for Hindi health-domain NER.

1) MuRIL Representation Layer: Given an input token sequence $X = \{x_1, x_2, \ldots, x_T\}$, MuRIL generates contextualized token representations using a transformer encoder. Instead of being dependent on a single layer, representations

from the final four transformer layers are added to capture both syntactic and semantic information:

$$E_i = \sum_{k=L-3}^{L} H_i^{(k)} \tag{1}$$

where $H_i^{(k)}$ denotes the hidden state of the i-th token from the k-th transformer layer, and L is the total number of MuRIL layers.

Here, x_i denotes the i-th input token, T is the sequence length, E_i is the final contextual representation of token i, and $H_i^{(k)} \in \mathbb{R}^{768}$ represents the hidden state vector produced by the k-th MuRIL layer.

2) CRF Decoding: At last, the CRF layer is put into use to learn relations between labels and to make sure that the predicted tag sequence is valid.Given emission scores P and transition matrix T, the accurate label sequence is decoded as:

$$Y^* = \arg\max_{Y} \sum_{i=1}^{T} \left(P_{i,y_i} + T_{y_{i-1},y_i} \right) \tag{2}$$

where y_0 denotes a special START tag.

In this equation, $Y = \{y_1, \ldots, y_T\}$ denotes a candidate label sequence, P_{i,y_i} is the emission score for assigning label y_i to token i, and T_{y_{i-1},y_i} models the transition score between successive labels.

B. Stacked MuRIL

This work proposes a stacked neural architecture consisting of MuRIL embeddings followed by BiLSTM, token-level attention, CNN, concatenation, fully connected, and CRF layers.

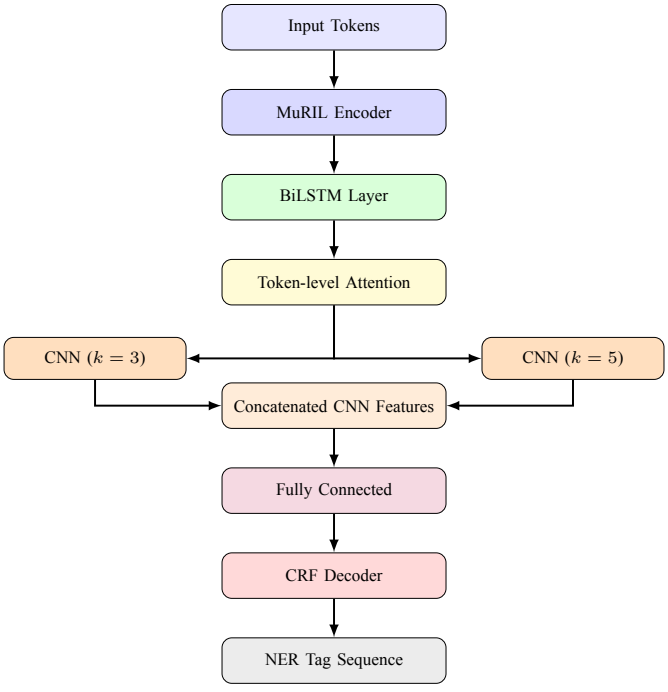

Fig. 2. Stacked MuRIL-based architecture integrating BiLSTM, token-level attention, CNN feature extraction, and CRF decoding for Hindi health-domain NER.

1) MuRIL Representation Layer: Given an input token sequence $X = \{x_1, x_2, \ldots, x_T\}$, MuRIL generates contextualized token representations using a transformer encoder. Instead of being dependent on a single layer, representations from the final four transformer layers are added to capture both syntactic and semantic information:

$$E_i = \sum_{k=L-3}^{L} H_i^{(k)} \tag{3}$$

where $H_i^{(k)}$ denotes the hidden state of the i-th token from the k-th transformer layer, and L is the total number of MuRIL layers.

Here, x_i denotes the i-th input token, T is the sequence length, E_i is the final contextual representation of token i, and $H_i^{(k)} \in \mathbb{R}^{768}$ represents the hidden state vector produced by the k-th MuRIL layer.

2) Bidirectional LSTM Layer: The sequence of MuRIL representations $E = \{E_1, E_2, \ldots, E_T\}$ is fed into a Bidirectional LSTM (BiLSTM) to capture model sequential relationships in both forward and backward directions:

$$H = \text{BiLSTM}(E) \tag{4}$$

At every time step i, the output representation is made by concatenating the forward and backward hidden states:

$$H_i = [\overrightarrow{h_i}; \overleftarrow{h_i}] \tag{5}$$

This layer helps transformer-based context modeling by clearly encoding ordering information.

In this formulation, $H = \{H_1, \ldots, H_T\}$ denotes the BiLSTM output sequence, $\overrightarrow{h_i}$ and $\overleftarrow{h_i}$ are the forward and backward hidden states at position i, and $[\cdot;\cdot]$ represents vector concatenation.

3) Token-level Attention Layer: To give importance to medically important tokens, a token-level attention mechanism is used over the BiLSTM outputs. Each token is given an importance weight computed as:

$$e_i = w^\top \tanh(W H_i) \tag{6}$$

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{T} \exp(e_j)} \tag{7}$$

$$A_i = \alpha_i H_i \tag{8}$$

where α_i represents the normalized attention weight for token i. This mechanism works as a soft gating function that highlights important medical entities such as symptoms, diseases, and medications.

Here, W and w are learnable attention parameters, e_i is the unnormalized attention score for token i, α_i is the normalized attention weight, and A_i denotes the attention-weighted token representation.

4) CNN and Fully Connected Layers: The attention-weighted sequence A is processed using multiple one-dimensional convolutional filters applied along the temporal

dimension to capture local n-gram patterns. For a convolutional filter with kernel size k, the operation is defined as:

$$C_k = \text{ReLU}(\text{Conv}_k(A)) \tag{9}$$

In this work, convolutional filters with kernel sizes $k \in \{3, 5\}$ are employed. The resulting feature maps are concatenated channel-wise:

$$C = \text{Concat}(C_3, C_5) \tag{10}$$

The concatenated features are then projected to token-level label emission scores by feeding it into a fully connected layer:

$$P_i = \text{FC}(C_i) \tag{11}$$

where $P_i \in \mathbb{R}^{|\mathcal{Y}|}$ represents the emission score vector over the label set $\mathcal{Y}$.

Here, $\text{Conv}_k(\cdot)$ denotes a 1D convolution with kernel size k, C_k is the corresponding feature map, C_i is the concatenated feature vector at token position i, and $\mathcal{Y}$ denotes the set of possible BIO labels.

5) CRF Decoding: At last, the CRF layer is put into use to learn relations between labels and to make sure that the predicted tag sequence is valid.Given emission scores P and transition matrix T, the accurate label sequence is decoded as:

$$Y^* = \arg\max_{Y} \sum_{i=1}^{T} \left(P_{i,y_i} + T_{y_{i-1},y_i} \right) \tag{12}$$

where y_0 denotes a special START tag.

In this equation, $Y = \{y_1, \ldots, y_T\}$ denotes a candidate label sequence, P_{i,y_i} is the emission score for assigning label y_i to token i, and T_{y_{i-1},y_i} models the transition score between successive labels.

V. Experimental Settings

This section describes the details of the dataset we used in our experiments, the preprocessing pipeline, its training details, and also the evaluation protocol followed for all models.

A. Dataset and Data Split

We performed all our experiments on the Hindi Health Dataset (HHD) [25], which is a collection of Hindi text related to healthcare tasks and is annotated at the token level for named entity recognition task. The dataset is annotated using the BIO (IOB2) tagging scheme. We have selected four medical entity categories—*Disease*, *Symptom*, *Consumable*, and *Person*—along with the non-entity label *O*.

For model training and evaluation, we randomly shuffled the sentences with a fixed seed value and split the dataset into **70%** training, **15%** validation, and **15%** testing sets. On doing so, we obtained 3259 sentences as training samples and 698 sentences as validation samples, with the remaining portion reserved for testing samples.

B. Preprocessing and Tokenization

All the input sentences are encoded using the MuRIL tokenizer. Since MuRIL performs subword tokenization, it is not always possible to have a direct one-to-one mapping between the input words and the labels. Therefore , inorder to handle this issue, we assign the original NER label only to the first sub-word token of each word and mark all remaining sub-words with the ignore index `-100`. These ignored sub-word positions are skipped during both loss computation and metric evaluation. During batching, sequences are padded and truncated to a fixed maximum length, and the attention mask is used to ensure that padded tokens do not influence model predictions.

C. Model Configurations

To study the impact of different MuRIL-based training and decoding stratergies, we compare several baseline configurations which are listed as follows :

- **MuRIL + CRF (Baseline):** The MuRIL model is fully fine-tuned and decoded using a CRF layer [24].
- **Frozen MuRIL Encoder + CRF:** The MuRIL encoder is left frozen, and only the CRF layer is trained.
- **MuRIL + Softmax (No CRF):** The MuRIL model is fine-tuned and the tokens are classified using a standard softmax classifier without CRF decoding.
- **MuRIL + LoRA + CRF:** Parameter-efficient fine-tuning is performed using LoRA adapters, followed by CRF decoding.
- **Stacked Architecture:** The MuRIL representations are enriched using BiLSTM, token-level attention, CNN-based feature extraction, and CRF decoding.

D. Training Setup

All models are implemented using the PyTorch Framework and are trained using the AdamW optimizer with weight decay. We train each model for a maximum of 15 epochs and employ early stopping based on the validation set F1-score with a patience value of 3 epochs. The best performing checkpoint on the validation set is then selected for the final test-set evaluation.All models were trained using the same data splits,same random seed and same evaluation protocol to ensure a fair comparison across variants. To stabilize the training process, the ReduceLROnPlateau learning-rate scheduler is used, which decreases the learning rate when the validation performance plateaus. Gradient clipping is also used to prevent unstable updates to the model weights.

For the CRF-based models, training is performed by optimizing the negative log-likelihood loss function, and the predictions are obtained using Viterbi decoding. The softmax baseline is trained using cross-entropy loss for token-level classification.

In terms of the trainable parameters, both the baseline model and the softmax baseline variant update all model parameters during fine-tuning, whereas the frozen-encoder setup trains only the CRF layer (7,020 trainable parameters) keeping MuRIL Encoder fixed.The stacked model introduces

additional trainable parameters from BiLSTM,token-level attention,CNN and CRF layers, while also allowing MuRIL weights to be updated. Finally, LoRA fine-tuning updates only 105,324 parameters (approximately 0.04% of the full model), making it a parameter-efficient alternative.

For the stacked model, to optimize different parts of the architecture effectively, we use separate learning rates for each module: 2×10^{-5} for the MuRIL encoder, 1×10^{-3} for BiLSTM/attention/CNN/fully connected layers, and 5×10^{-4} for the CRF layer.

E. Evaluation Metrics

We report entity-level Precision, Recall, and F1-score as the primary evaluation metrics. All results are evaluated under the strict IOB2 evaluation protocol using the `seqeval` framework. We ignore Padding tokens and subword positions (marked as `-100`) are excluded from evaluation to ensure fair comparison between models.

VI. Experimental Results

We evaluate all the models on the HHD test set using strict IOB2 entity-level Precision, Recall, and F1-score.

Model Architecture	Precision	Recall	F1-score
MuRIL + CRF (Baseline)	0.9509	0.9319	0.9413
MuRIL (Frozen) + CRF	0.8071	0.8209	0.8139
MuRIL + Softmax (No CRF)	0.9083	0.9376	0.9232
MuRIL + LoRA + CRF	0.8002	0.7853	0.7927
MuRIL + BiLSTM + Attention + CNN + CRF(Stacked)	0.9415	0.9415	0.9415

TABLE I
Overall Precision, Recall, and F1-score comparison of MuRIL-based NER models on the HHD test set.

From Table I, we observe that the baseline MuRIL+CRF model performs well on the HHD test set (F1 = 0.9413), which shows that fine-tuning the entire encoder, combined with CRF decoding is effective for healthcare-domain Hindi NER. The stacked model performs equally well with a overall (F1 = 0.9415), which suggests that adding sequential and local feature modeling layers can be helpful although the gain remains marginal at the overall level. There is a significant drop in the case of the frozen-encoder CRF model (F1 = 0.8139), which highlights the importance of encoder adaptation to domain-specific entities and vocabulary. The softmax baseline performs well in terms of recall (0.9376) but it has lower overall F1 score as compared to CRF-based decoding model. This shows that the CRF layer is helpful in maintining more consistent predictions for BIO-tagged sequences. Although LoRA fine-tuning is a parameter-efficient, updating only a small fraction of parameters leads to weaker precision and recall compared to full fine-tuning on this dataset for this particular task.

Model	Consumable	Disease	Person	Symptom
MuRIL + CRF (Baseline)	0.93	0.94	0.98	0.93
MuRIL (Frozen) + CRF	0.81	0.77	0.97	0.80
MuRIL + Softmax (No CRF)	0.91	0.92	0.97	0.91
MuRIL + LoRA + CRF	0.79	0.76	0.93	0.76
MuRIL + BiLSTM + Attention + CNN + CRF (Stacked)	0.93	0.96	0.98	0.94

TABLE II
Category-wise F1-score comparison across different models on the HHD test set.

From Table II, we observe that the baseline, softmax, and stacked models have stable performance across categories or entity types, whereas the frozen-encoder and LoRA-based settings show consistent degradation across most categories. The stacked model shows a slight improvement for *Disease* (0.96) and *Symptom* (0.94), which may be due to the additional layers capturing local patterns and sequential dependencies useful for healthcare entities. From the results, it appears that the full fine-tuning of MuRIL, along with structured CRF decoding, remains a strong choice for this dataset, while stacked feature augmentation provides a small improvements in specific categories.

VII. Conclusion

In this study, we conducted a detailed evaluation of multiple MuRIL-based architectures on the Hindi Health Domain Dataset out of which best F1 score was given by the Stacked MuRIL architecture. The baseline MuRIL–CRF architecture was only marginally behind suggesting that MuRIL contextual representation already capture much of the task-relevant information required for effective medical NER in Hindi.

Our analysis gives rise to an interesting observation that while architectural augmentations such as BiLSTM, attention, and CNNs can slightly improve model performance, the gains are marginal when compared to added model complexity, training costs and overparameterization.These findings demonstrate that, for Hindi healthcare NER, careful fine-tuning of a transformer–CRF architecture can be as effective as deeper stacked models. Overall, this empirical study provides practical insights into the trade-offs between architectural complexity and performance.

References

[1] Nadeau, D., Sekine, S.: A survey of named entity recognition and classification. Lingvisticae Investigationes 30(1), 3–26 (2007)
[2] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proc. NAACL-HLT. pp. 4171–4186 (2019)
[3] Liu, Y., et al.: RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692 (2019)
[4] Clark, K., Luong, M.-T., Le, Q.V., Manning, C.D.: ELECTRA: Pre-training text encoders as discriminators rather than generators. In: Proc. ICLR (2020)
[5] Lan, Z., et al.: ALBERT: A lite BERT for self-supervised learning of language representations. In: Proc. ICLR (2020)
[6] Raffel, C., et al.: Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR 21(140), 1–67 (2020)
[7] Lample, G., Conneau, A.: Cross-lingual language model pretraining. In: Proc. NeurIPS. pp. 7059–7069 (2019)
[8] Conneau, A., et al.: Unsupervised cross-lingual representation learning at scale. In: Proc. ACL. pp. 8440–8451 (2019)

[9] Khanuja, S., et al.: MuRIL: Multilingual representations for Indian languages. In: Proc. EMNLP. pp. 974–985 (2021)
[10] Athavale, V., et al.: Named entity recognition for Hindi using deep learning techniques. In: Proc. ICON (2016)
[11] Gupta, P., et al.: Deep learning-based named entity recognition for code-mixed Indian social media text. In: Proc. COLING. pp. 51–62 (2018)
[12] Murthy, R.: Named entity recognition for Indian languages using neural networks. M.Tech Thesis, IIIT Hyderabad (2017)
[13] A. P., S., et al.: A BiLSTM-CRF based approach for Hindi named entity recognition. In: Proc. FIRE (2019)
[14] Sharma, N., et al.: Improving Hindi named entity recognition using CRF and deep learning approaches. Expert Systems with Applications 147, 113204 (2020)
[15] Shah, S., Kopparapu, S.K.: Denoising autoencoders for improved Hindi NER. In: Proc. ICON (2019)
[16] Kakwani, D., et al.: IndicBERT: A pre-trained language model for Indian languages. arXiv preprint arXiv:2005.00085 (2020)
[17] Souza, F., et al.: Portuguese named entity recognition using BERT-CRF. In: Proc. BRACIS (2019)
[18] Hakala, K., Pyysalo, S.: Biomedical named entity recognition with multilingual BERT. In: Proc. BioNLP Workshop (2019)
[19] Cui, Y., et al.: Fine-tuning BERT for Chinese named entity recognition. arXiv preprint arXiv:1905.05526 (2019)
[20] Mukhin, V.: Russian named entity recognition using transformer models. In: Proc. AIST (2020)
[21] Labusch, K., et al.: German NER using pre-trained transformer models. In: Proc. KONVENS (2020)
[22] Arkhipov, M., et al.: Named entity recognition for Slavic languages using BERT. In: Proc. RANLP (2019)
[23] Xue, L., et al.: mT5: A massively multilingual pre-trained text-to-text transformer. In: Proc. NAACL-HLT. pp. 483–498 (2020)
[24] R. Sharma, S. Morwal, and B. Agarwal, *Named entity recognition using neural language model and CRF for Hindi language*, Computer Speech & Language, vol. 74, p. 101356, 2022.
[25] A. Jain, "Hindi Health Dataset," Kaggle. Available at: https://www.kaggle.com/datasets/aijain/hindi-health-dataset